

TRÌNH BIÊN DỊCH

Chương 3. Dịch trực tiếp cú pháp

Phạm Nguyễn Khang
BM. Khoa học máy tính

pnkhang@cit.ctu.edu.vn

CẦN THƠ, 12 - 2012

```
#include ...  
int main() {  
    ...  
}
```



Phân tích từ vựng

Phân tích cú pháp

Phân tích ngữ nghĩa

Sinh mã trung gian

Tối ưu mã trung gian

Sinh mã

Tối ưu



```
0001001000  
1011011100  
1100101111  
1000011111
```

Dịch trực tiếp cú pháp (Syntax-directed translation)

- Dịch trực tiếp cú pháp/dịch hướng cú pháp
 - là một phương pháp cài đặt trình biên dịch trong đó việc dịch chương trình nguồn được điều khiển (drive) bằng bộ phân tích cú pháp.
 - Quá trình phân tích cú pháp và cây phân tích cú pháp được dùng để điều khiển việc phân tích ngữ nghĩa và dịch chương trình nguồn.
 - Có thể thực hiện việc dịch này bằng 2 cách:
 - Phân tích cú pháp → Cây phân tích cú pháp → Phân tích ngữ nghĩa
 - Bổ sung các thông tin điều khiển phân tích ngữ nghĩa (hành vi ngữ nghĩa) vào văn phạm → **Văn phạm thuộc tính (Attribute grammar)**

Văn phạm thuộc tính

- Văn phạm được tăng cường bằng cách:
 - Thêm các **thuộc tính (attributes)** cho các ký hiệu chưa kết thúc/kết thúc của văn phạm để mô tả tính chất của nó.
 - Mỗi thuộc tính có tên và giá trị được gán cho nó. Ví dụ: chuỗi, số, kiểu, vị trí trong bộ nhớ, số hiệu thanh ghi, ... bất kể là thông tin gì mà ta muốn.
 - Ví dụ: một biến (variable) có thể có thuộc tính kiểu để kiểm tra kiểu trong biểu thức.

Văn phạm thuộc tính

- Với mỗi luật sinh ta thêm vào các luật ngữ nghĩa gọi là các hành động (actions), mô tả cách tính giá trị của các thuộc tính.
- Giá trị thuộc tính có thể được tính từ giá trị của các nút con, nút anh em hay nút cha của nó.

- Ví dụ:

```
digit -> 0 {digit.value = 0}  
        | 1 {digit.value = 1}  
        | 2 {digit.value = 2}  
        ...  
        | 9 {digit.value = 9}
```

Văn phạm thuộc tính

- Thuộc tính có thể được truyền trong cây để tính giá trị cho các thuộc tính khác:

$int_1 \rightarrow digit \{int_1.value = digit.value\}$
| $int_2 digit \{int_1.value = int_2.value * 10 +$
 $digit.value\}$

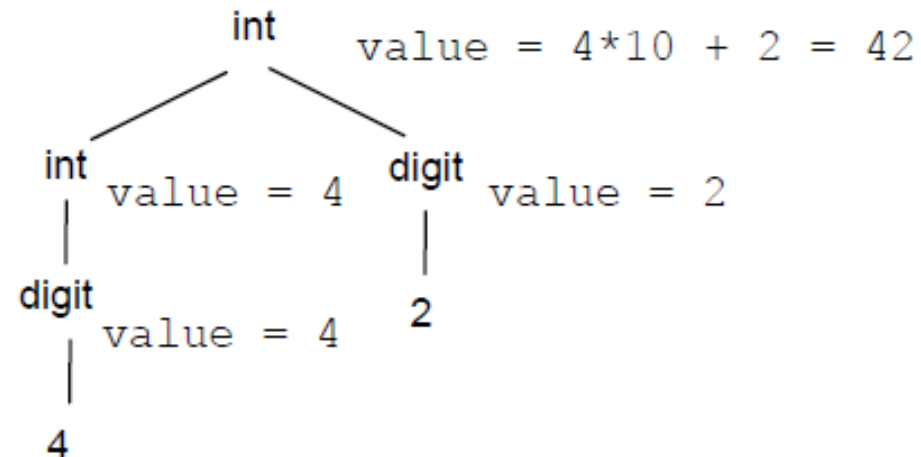
- Chỉ số dưới ví dụ: int_1 , int_2 được sử dụng để phân biệt biến xuất hiện nhiều lần trong luật sinh.

Thuộc tính tổng hợp và thuộc tính kế thừa

- Có 2 loại thuộc tính:
 - Tổng hợp (synthesized)
 - Kế thừa (inherited)
- Thuộc tính tổng hợp:
 - Truyền ngược lên trên (nút con $\rightarrow$ nút cha)
 - Ví dụ: thuộc tính của ký hiệu về trái được tính từ các thuộc tính của các ký hiệu bên về phải của luật sinh.
 - Bộ phân tích từ vựng (lexical analyser) thường cung cấp các thuộc tính cho các ký hiệu kết thúc.

$X \rightarrow Y_1 Y_2 \dots Y_n$

$X.a = f(Y_1.a, Y_2.a, \dots Y_n.a)$



Thuộc tính tổng hợp và thuộc tính kế thừa

- Thuộc tính kế thừa:
 - Truyền xuống (nút cha $\rightarrow$ nút con) hoặc truyền ngang

$$X \rightarrow Y_1 Y_2 \dots Y_n$$

$$Y_k.a = f(X.a, Y_1.a, Y_2.a, \dots, Y_{k-1}.a, Y_{k+1}.a, \dots, Y_n.a)$$

- Thuộc tính này thường dùng để truyền thông tin ngữ cảnh của nút hiện tại

Thuộc tính tổng hợp và thuộc tính kế thừa

- Xét văn phạm:
 - $P \rightarrow DS$
 - $D \rightarrow \text{var } V; D \mid \varepsilon$
 - $S \rightarrow V := E; S \mid \varepsilon$
 - $V \rightarrow x \mid y \mid z$
- Ta thêm 2 thuộc tính: **name** (tên biến) và **d1** (danh sách biến được khai báo).
- Mỗi khi khai báo biến mới, thuộc tính tổng hợp **name** được gán bằng giá trị của biến mới. Thêm tên mới này vào danh sách biến **d1**.

Thuộc tính tổng hợp và thuộc tính kế thừa

- Văn phạm tăng cường như sau:

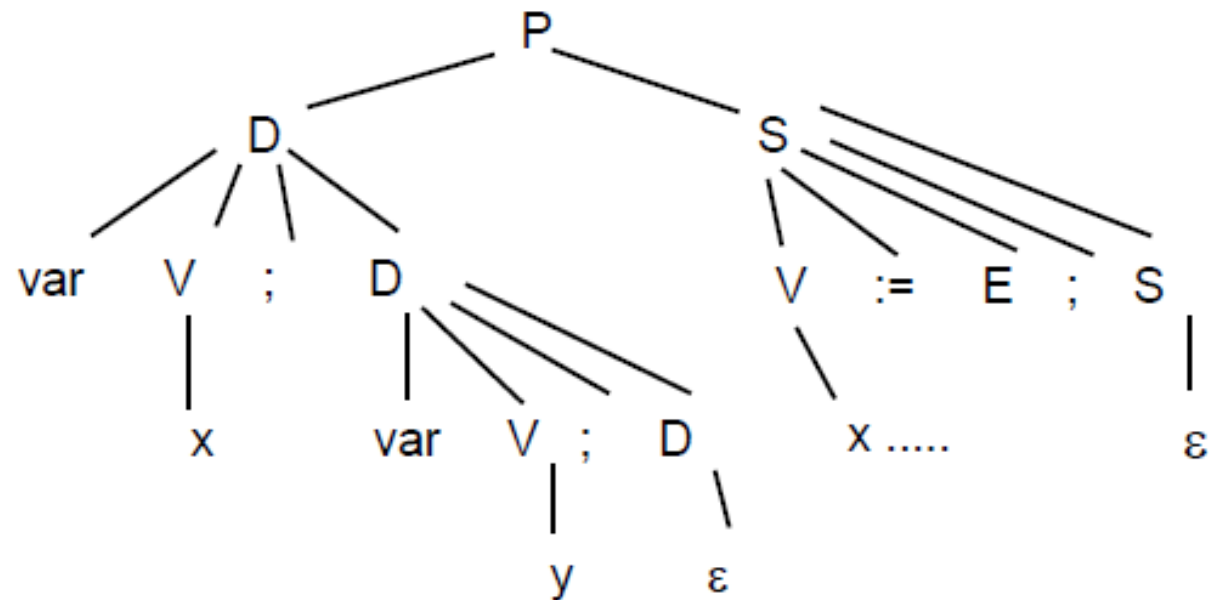
P	→	DS	{ S.dl = D.dl }
D ₁	→	var V; D ₂	{ D ₁ .dl = addlist(V.name, D ₂ .dl) }
		ε	{ D ₁ .dl = NULL }
S ₁	→	V := E; S ₂	{ check(V.name, S ₁ .dl); S ₂ .dl = S ₁ .dl }
		ε	
V	→	x	{ V.name = 'x' }
		y	{ V.name = 'y' }
		z	{ V.name = 'z' }

Thuộc tính tổng hợp và thuộc tính kế thừa

- Với chuỗi nhập này thì các thuộc tính được truyền như thế nào ?

```
var x;  
var y;
```

```
x := ...;  
y := ...;
```



Thuộc tính tổng hợp và thuộc tính kế thừa

```
typedef struct _attribute {  
    char *name;  
    struct _attribute *list;  
} attribute;
```

```
P -> DS          { $2.list = $1.list }  
D -> var V; D    { $$$.list = add_to_list($2.name, $4.list) }  
    | ε          { $$$.list = NULL }  
S -> V := E; S   { check($1.name, $$$.list); $5.list = $$$.list }  
    | ε  
V -> x           { $$$.name = 'x' }  
    | y          { $$$.name = 'y' }  
    | z          { $$$.name = 'z' }
```

\$\$: biến vế trái

\$i: token thứ i trong vế phải, bắt đầu từ 1

Dịch trực tiếp cú pháp từ trên xuống

- **Thuộc tính tổng hợp** được tính sau khi gọi hàm tương ứng có trong luật sinh.

- Ví dụ:

$E \rightarrow (E + E) \{ \$$.val = \$2.val + \$4.val; \}$
 $\quad \quad \quad | \text{ int } \{ \$$.val = \$1.val; \}$

```
int func_E() {  
    int lookahead = next_token();  
    if (lookahead == T_INT)  
        return val; //giá trị do bộ PTTV cung cấp  
    int val2 = func_E();  
    match('+');  
    int val4 = func_E();  
    match(')');  
    return val2 + val4;  
}
```

Dịch trực tiếp cú pháp từ trên xuống

- **Thuộc tính kế thừa** được truyền khi gọi hàm tương ứng có trong luật sinh.

- Ví dụ:

$E \rightarrow \text{int } \{\$2.\text{left} = \$1.\text{val}\} \quad R \{\$\$.val = \$2.val\}$

$R \rightarrow + \text{int } \{\$3.\text{left} = \$\$.left + \$2.val;\}$

$R \{\$\$.val = \$3.val;\}$

$| \varepsilon \{\$\$.val = \$\$.left;\}$

Dịch trực tiếp cú pháp từ trên xuống

```
int R_func(int);  
int E_func() {  
    int lookahead = yylex();  
    if (lookahead == T_INT) {  
        return R_func(val);  
    }  
    printf("syntax error\n");  
    return -1;  
}
```

Dịch trực tiếp cú pháp từ trên xuống

```
int R_func(int left) {  
    int lookahead = yylex();  
    if (lookahead == '+') {  
        lookahead = yylex();  
        if (lookahead == T_INT) {  
            return R_func(left + val);  
        }  
    }  
    return left;  
}
```


Dịch trực tiếp cú pháp từ dưới lên

- Văn phạm biểu thức số học:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow \text{int} \mid (E)$

- Văn phạm tăng cường (với luật ngữ nghĩa):

```
S : E {printf("KQ: %d.\n", $1);}  
;
```

```
E : E '+' T {$$ = $1 + $3;}  
  | T {$$ = $1;}  
;
```

```
T : T '*' F {$$ = $1 * $3;}  
  | F {$$ = $1;}  
;
```

```
F : T_INT {$$ = $1;}  
  | '(' E ')' {$$ = $2;}  
;
```

Giới thiệu Bison



- Bison là chương trình sinh ra bộ phân tích cú pháp (parser generator) sử dụng giải thuật LALR(1).
- Tiền thân của Bison là Yacc (yet another compiler compiler)



Yak



Bison

Bison

- Hoạt động như thế nào ?
 - Bison được thiết kế để làm việc với ngôn ngữ C (có thể mở rộng cho C++)
 - Có thể kết hợp với flex để xây dựng các trình biên dịch hoàn chỉnh
 - Biên dịch:

```
$ bison myFile.y
```

```
$ gcc -c myFile.tab.c
```

```
$ gcc -o parse myFile.tab.o lex.yy.o -lfl -ly
```

```
$ ./parse
```

Bison

- Cấu trúc file .y:

%{

Khai báo

%}

Định nghĩa

%%

Các luật sinh

%%

Các hàm/thủ tục do người dùng định nghĩa

Ví dụ exp.y

```
%{  
    #include <stdio.h>  
%}  
  
%left '+'  
%left '*'  
%token T_INT
```

```
%%  
E : E '+' T {$$ = $1 + $3;}  
  | T {$$ = $1;}  
;  
T : T '*' F {$$ = $1 * $3;}  
  | F {$$ = $1;}  
;  
F : T_INT {$$ = $1;}  
  | '(' E ')' {$$ = $2;}  
;  
%%  
  
int main() {  
    yyparse();  
    return 0;  
}
```

Cấu trúc file .y

- Thuộc tính của các kí hiệu được bộ phân tích từ vựng (flex) gán thông qua biến **yylval**.

%%

```
[0-9]+ { yyval =  
    atoi(yytext);  
    return T_INT; }
```

...

- Với mỗi luật sinh, ta sử dụng
 - Ta sử dụng dấu hai chấm (:) thay cho dấu mũi tên,
 - Dấu | để phân cách các luật sinh có cùng vế trái
 - Dấu chấm phẩy (;) kết thúc luật sinh.
 - Luật ngữ nghĩa được đặt trong cặp dấu ngoặc nhọn.

- Luật sinh đầu tiên được giả định là luật sinh bắt đầu của văn phạm.
- Hàm yyparse là hàm chính của bison để thực hiện việc phân tích cú pháp.
- Nếu có lỗi xảy ra hàm yyerror sẽ được gọi. Ta có thể định nghĩa lại hàm này

Token, luật sinh và hành động

- Token được định nghĩa bằng chỉ thị %token, ví dụ:

```
%token T_Int T_Double T_String T_While
```

- Luật sinh và hành động được khai báo cùng lúc:

```
left_side: right_side1 { action1 }  
| right_side2 { action2 }  
| right_side3 { action3 }  
;
```

- left_side là kí hiệu chưa kết thúc nằm ở vế trái
- right_side là vế phải
- Phần nằm trong cặp dấu ngoặc nhọn là các hành động.

Thuộc tính của các ký hiệu

- Bộ phân tích cú pháp cho phép sử dụng thuộc tính cho các ký hiệu có trong văn phạm.
- Biến **yylval** được dùng để truyền giá trị từ bộ phân tích từ vựng cho bộ phân tích cú pháp. Mặc định biến này có kiểu là **int**.
- Để sử dụng các thuộc tính khác nhau cho các ký hiệu khác nhau ta có thể sử dụng chỉ thị **%union**. Ví dụ:

```
%union {  
    int intValue;  
    double doubleValue;  
    char *stringValue;  
}  
%token <intValue>T_Int <doubleValue>T_Double  
%token <stringValue>T_String  
%type<intValue> Integer IntExpression
```


Thuộc tính của kí hiệu

- Để truy xuất thuộc tính của các ký hiệu trong hành động của luật sinh ta dùng cú pháp:
 - \$\$: thuộc tính của vế trái
 - \$i: thuộc tính của kí hiệu thứ i trong vế phải, bắt đầu từ 1.

Giải quyết độ bằng khai báo độ ưu tiên

- Xét văn phạm sau:

```
%token T_Int
%%
E : E '+' E
  | E '*' E
  | E '^' E
  | T_Int
;
```

- Bison cho ra 9 conflicts shift/reduce

- Khai báo độ ưu tiên

```
%token T_Int
%left '+'
%left '*'
%right '^'
%%
E : E '+' E
  | E '*' E
  | E '^' E
  | T_Int
;
```

Giải quyết độ bằng khai báo độ ưu tiên

- Khai báo độ ưu tiên cho ký hiệu kết thúc
 - Dùng chỉ thị `%left`, `%right` hoặc `%nonassoc`
 - Có thể khai báo nhiều kí hiệu kết thúc trên cùng một hàng. Chúng sẽ có độ ưu tiên bằng nhau, ví dụ:

```
%left '+' '-'  
%left '*' '/'  
%right '^'
```
 - Cái nào khai báo sau sẽ có độ ưu tiên cao hơn.

Giải quyết đụng độ bằng khai báo độ ưu tiên

- Giải quyết đụng độ như thế nào ?
 - Khi có đụng độ shift/reduce xảy ra, bison sẽ dựa trên **độ ưu tiên của ký hiệu lookahead** và **độ ưu tiên của luật sinh sẽ rút gọn**, cái nào ưu tiên cao hơn sẽ được thực hiện.
 - Độ ưu tiên của luật sinh là độ ưu tiên của ký hiệu kết thúc phải nhất có trong luật sinh.
 - Ta cũng có thể định nghĩa độ ưu tiên của luật sinh bằng chỉ thị %prec
 - Được sử dụng khi vế phải của luật sinh không có ký hiệu kết thúc hoặc
 - Khi ta muốn định nghĩa chồng lên độ ưu tiên của ký hiệu kết thúc phải nhất

Định nghĩa độ ưu tiên cho luật sinh

`%left '+' '-'`

`%left '*' '/'`

`%nonassoc UMINUS`

`E : E '+' E`

`| E '-' E`

`| E '*' E`

`| E '/' E`

`| '(' E ')'`

`| '-' E %prec UMINUS`

Luật sinh này sẽ có độ ưu tiên cao nhất

Nếu không sử dụng `%prec`, luật sinh này sẽ có độ ưu tiên bằng với phép trừ hai ngôi.

Xử lý lỗi

- Khi bison phát hiện ra lỗi cú pháp, nó gọi hàm `yyerror()`. Mặc định hàm này in ra một câu thông báo lỗi và dừng lại.
- Bison hỗ trợ cơ chế đồng bộ hóa lại lỗi (error re-synchronization) cho phép bỏ qua lỗi và tiếp tục việc phân tích.
- Sử dụng token đặc biệt **error** bên vế phải của luật sinh cho phép phục hồi lỗi.
- Ví dụ:

```
Var : Modifiers Type IdentifierList ';'
    | error ';'
    ;
```
- Nếu có lỗi xảy ra khi phân tích `Var`, bộ phân tích sẽ bỏ qua các ký hiệu vào stack cho đến khi gặp token `';`', sau đó reduce **error `';`'** về **Var**

Demo

- Xây dựng trình biên dịch cho các biểu thức số học với văn phạm:
 - $E \rightarrow E '+' E$
 - $E \rightarrow E '-' E$
 - $E \rightarrow E '*' E$
 - $E \rightarrow E '/' E$
 - $E \rightarrow (E)$
 - $E \rightarrow '-' E$
 - $E \rightarrow \text{int}$
- Độ ưu tiên toán tử từ thấp đến cao:
 - $'+', '-'$
 - $*, /$
 - $'-'$ một ngôi

Tập tin exp.lex

```

1  %{
2  #include "exp.tab.h"
3  %}
4  %%
5  [0-9]+  {
6           |      yyval = atoi(yytext);
7           |      return T_INT;
8           }
9  ")"  "      return ')';
10 "("  "      return '(';
11 "+"  "      return '+';
12 "-"  "      return '-';
13 "*"  "      return '*';
14 "/"  "      return '/';
15 .
16 %%

```


Tập tin exp.y

```
1 %{
2 #include <stdio.h>
3 %}
4 %token T_INT
5 %left '+' '-'
6 %left '*' '/'
7 %nonassoc UMINUS
8 %%
9 S : E {printf("KQ: %d", $1);}
10 E : E '+' E {$$ = $1 + $3;}
11   | E '-' E {$$ = $1 - $3;}
12   | E '*' E {$$ = $1 * $3;}
13   | E '/' E {$$ = $1 / $3;}
14   | '(' E ')' {$$ = $2;}
15   | '-' E %prec UMINUS {$$ = -$2;}
16   | T_INT {$$ = $1;}
17 ;
18 %%
```

```
19 int main() {
20     yyparse();
21     return 0;
22 }
```

Biên dịch & thực thi

- **Biên dịch**

- `flex exp.lex`
- `bison -d exp.y`
- `gcc exp.tab.c yy.lex.c -lfl -ly -o exp`

- **Chạy chương trình:**

- `echo "4 + 5 * 2" | exp`
- **Cho kết quả:**
KQ: 14

Bài tập lập trình 1

- Cải tiến ví dụ demo
 1. Thêm vào phép toán lũy thừa '^' có độ ưu tiên cao hơn phép '*' nhưng thấp hơn UMINUS, kết hợp phải
 2. Thêm vào luật sinh cho phép chương trình nguồn viết mỗi biểu thức trên 1 dòng. Khi biên dịch chương trình sẽ tính toán giá trị của từng biểu thức và in kết quả theo từng dòng. Ví dụ với tập tin đầu vào:
 - $4 + 3 * 2$
 - $7 * 6 / 3$Sẽ cho kết quả
 - 10
 - 14

Bài tập lập trình 1

3. Xử lý lỗi

- Thông báo lỗi cho các dòng có lỗi

Ví dụ:

4 + 3 * 2

5 * +

7 * 6 / 3

Sẽ cho kết quả

10

error

14

4. Xử lý dòng trống

- In câu bỏ qua cho các dòng trống

Ví dụ:

4 + 3 * 2

5 * +

7 * 6 / 3

Sẽ cho kết quả

10

Error

Bo qua

14

5. Cho phép dòng cuối cùng không cần dấu xuống dòng.

Gợi ý

1. Định nghĩa thêm hàm **luy_thua** bên dưới phần khai báo luật sinh (phía sau dấu %%).
2. Thêm token '\n' trong file exp.lex và định nghĩa lại văn phạm cho phù hợp với yêu cầu.

`line → E '\n'`

3. Sử dụng token error
4. Thêm luật sinh rỗng

`line → '\n'`

5. Xử lý file exp.lex

- Sử dụng luật <<EOF>> khi hết file mà trước đó không phải là dấu xuống dòng thì trả về dấu xuống dòng, lần thứ hai mới trả về 0.

Yêu cầu

- Mỗi nhóm: 2 SV
- Trình bày:
 - Chuẩn bị slides trình bày các công việc đã thực hiện
 - Demo chương trình kết quả
- Thời gian: 3 tuần